

Теми за Държавен Изпит

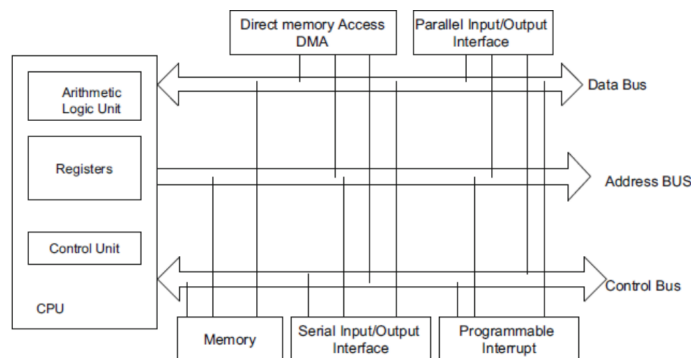
Д. Иванов, КН

11 юли 2025 г.

Компютърни архитектури

9. Компютърни архитектури. Формати на данните. Вътрешна структура на централен процесор – блокове и конвейерна обработка, инструкции.

Обща структура на компютрите и концептуално изпълнение на инструкциите, запомнена програма



1. **Централен процесор (CPU)** - основният елемент, който приема данни от **входните устройства**, обработва ги в информация, която трансферира в **паметта** и **изходните устройства**. Състои се от три основни блока - **аритметично и логическо устройство (ALU)**, **регистри** и **контролен блок**. В ALU се извършва набор от аритметични и логически операции, регистрите са оформени като малка многопортова памет, а контролният блок управлява входно/изходните устройства, генерира управляващи сигнали към останалите компоненти на системата, като сигнали за четене/запис и организира изпълнението на инструкциите.
2. **Памет** - състои се от оперативна памет **RAM** (Random-Access Memory) за временно съхранение на данни и програмен код и **ROM** (Read-Only

Memory) за съхраняване на системен код и информация.

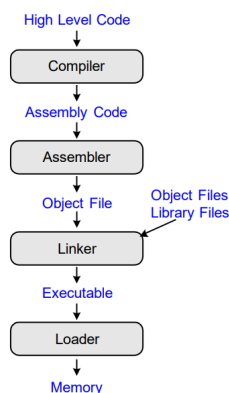
3. **Системна шина** - състои се от **адресна шина**, **шина данни** и **контролна шина**, която включва сигнали за синхронизация и управление и всички други сигнали, които не са адреси и данни.
4. Схема за **директен достъп до паметта (DMA)** - позволява обмен на данни между паметта и други устройства, без участието на процесора. При персоналните компютри е част от Северният мост (**North Bridge**)
5. **Многоканален входно/изходен контролер (I/O Controller)** - осъществява интерфейс с външни устройства, както и с други входно/изходни устройства като клавиатура и мишка. Част от Южния мост (**South Bridge**)
6. **Програмируем контролер на прекъсванията (Programmable Interrupt Controller)** - осъществява синхронизацията на заявките за прекъсване от различните входно/изходни устройства, като проверява текущия им статус. При персоналните компютри е част от Южния мост.

Изпълнение на инструкциите – може да се раздели на няколко стъпки:

1. Извличане (**Fetch**) на инструкция от паметта и вкарване в CPU;
2. Декодиране (**Decode**) на инструкцията – определяне на типа на инструкцията и участващите операнди от контролния блок на процесора.
3. Изпълнение (**Execute**) на инструкцията – подаване на операндите към АЛУ и извършване на операция с тях.
4. Запис на резултата от АЛУ в паметта (**Memory**).
5. Запис на резултата от АЛУ в регистър на процесора (**Writeback**), ако това е необходимо.

Запомнена програма – програмите се зареждат в паметта на компютъра на определен от архитектурата на системата начален адрес, като това става на няколко стъпки:

1. Език от високо ниво → Компилятор → Асемблерски код
2. Асемблерски код → Асемблер → Обектен файл
3. Обектен файл + Библиотечни файлове → Линкер → Изпълним код
4. Изпълним код → Лоудър → Зареден в паметта програмен код



Форматите на данните са начинът, по който числата и информацията се представят и съхраняват в компютърните системи. Те играят ключова роля в обработката на данни и в това как информацията се предава между различните части на системата.

1. **Цели двоични числа** (binary integers) - представени са в двоичен формат (с основа 2). В този формат числата се съхраняват като поредица от битове (0 и 1). Числото 5 в двоична система е представено като 101. В 8-битов формат това ще бъде 00000101
2. **Двоично-десетични числа** (binary-coded decimals) - този формат използва 4 бита за представяне на всяка цифра от десетичното число. Всяка десетична цифра се преобразува в съответното ѝ двоично представяне. Десетичното число 45 се представя като 0100 0101 в BCD формат, където 0100 е 4 и 0101 е 5
3. **Двоични числа с плаваща запетая** (floating-point numbers) - използват се за представяне на реални числа, които могат да имат дробна част. Те се състоят от три части: знак (за положителност и отрицателност), мантиса и експонента. Най-често се използва стандартът IEEE 754, където числата се представят с 32-бита или 64-бита. Числото 3.14 в 32-битов формат ще бъде представено като 0 10000000 10010001111010111000011, където първият бит е за знак, следващите 8 бита за експонента и останалите за мантисата.
4. **Знакови данни** (signed data) - това са числа, които могат да бъдат положителни или отрицателни. В двоичния формат първия бит обикновено се използва за знак (0 за положително, а 1 за отрицателно). В 8-битов знаков формат числото -5 се представя като 11111011
5. **Кодови таблици** (code tables) - използват се за представяне на символи и други специални знаци в компютрите. Най-известните кодови таблици включват ASCII и Unicode. ASCII стандартът представя символи чрез 7 или 8 бита, където всеки символ има уникален двоичен код. Unicode стандартът разширява ASCII, като поддържа много по-

вече символи и е пригоден за представяне на текстове на различни езици. Той използва различни кодировки като UTF-8, UTF-16, UTF-32

Централен процесор

Централният процесор (CPU) е основният компонент на компютърната система, който изпълнява инструкциите на програмите чрез извършване на основни аритметични, логически, контролни и входно-изходни операции. Нека разгледаме основните компоненти на CPU и тяхната роля.

Регистри - малки, високоскоростни запамятаващи устройства в CPU, които съхраняват данни и инструкции, които се използват от процесора по време на изпълнение. Те са изключително бързи, но с ограничен капацитет. Видове регистри:

- Акумулатор (accumulator) - използва се за съхранение на междинни резултати от изчисления
- Регистър за инструкции (instruction register) - съдържа текущата инструкция, която трябва да бъде изпълнена
- Базов регистър (base register) - съхранява началния адрес на сегмент в паметта
- Регистър на програмния брояч (program counter) - показва адреса на следващата инструкция, която ще бъде изпълнена

Аритметично-логическо устройство (ALU) - компонентът от централния процесор, който извършва аритметични операции като събиране, изваждане, умножение и деление и логически операции като сравнения и побитови операции. АЛУ обработва данни от регистрите и връща резултатите обратно в регистрите или в паметта

Регистри на състоянието и флаговете (status registers and flags) - тези регистри съдържат информация за състоянието на процесора и резултатите от последната операция на АЛУ. Флагове:

- Zero Flag (Z) - показва дали резултатът от операцията е нула
- Carry Flag (C) - показва дали има пренос или заем от аритметична операция
- Sign Flag (S) - показва знака на резултата (положителен или отрицателен)
- Overflow Flag (O) - показва дали е настъпило препълване при аритметична операция

Блокове за управление (control unit) - координира и контролира всички операции на CPU, като дешифрира инструкциите и генерира контролни

сигнали за другите компоненти. Основните му функции са:

- декодиране на инструкциите (instruction decoding) - разбира и интерпретира инструкциите, зададени от програмите
- генериране на контролни сигнали - насочва другите компоненти как да изпълняват инструкциите

Връзка с паметта (memory interface) - централният процесор взаимодейства с основната памет (RAM) чрез шините за адреси, данни и управление. Чрез тези шини се извършва четене и запис на данни от и към паметта. Основните функции тук са:

- адресиране на паметта - определя адреса на данните, които трябва да бъдат извлечени или записани
- извличане на данни - получава данни от паметта за обработка
- запис на данни - поставя резултати от обработката обратно в паметта

Дешифриране на инструкциите (instruction decoding) - процес, който включва интерпретация на инструкциите, които са извлечени от паметта, и превръщането им в сигнали, които контролират различните части на CPU. Този процес включва идентифициране на типа операция, адресирането на операндите и генериране на съответните контролни сигнали

Преходи (control flow) - отнасят се до смяната на изпълняваните инструкции в зависимост от условията в програмата. Това включва разклонения, цикли и условни операции. Видове преходи са:

- последователни - инструкциите се изпълняват последователно една след друга
- условни - програмният поток може да се промени в зависимост от определени условия (if-else)
- непосредствени - програмата директно прескача на ново място в паметта (goto)

10. Структура и йерархия на паметта. Сегментна и странична преадресация. Система за прекъсване – приоритети и обслужване.

Структура и йерархия на паметта

Памет в персоналния компютър се нарича всеки ресурс, който има свойството да съхранява информация във времето. Паметта представлява съвкупност от битове - информация, която се съхранява и има стойност 0 или 1. С развитието на компютрите се оказва, че най-удобната адресируема единица

е байтът, който представя осем бита. Паметта е организирана в клетки, наредени последователно една след друга, т.е. като едномерен линеен масив от байтове, които се номерират с последователни номера, наречени адреси. Номерацията започва от 0 и стига до последния наличен физически адрес. Паметта се изгражда като устройство, което може да извършва следните операции:

- четене - подаване на адрес и извеждане съдържанието на съответния байт
- писане - подаване на адрес и байт, който се записва на този адрес

Това е логическият модел на паметта. От архитектурна гледна точка, паметта се реализира физически като тримерни матрици. Когато се подаде двоичен адрес, има устройство което го дешифрира - адресът насочва устройството към елементите, които съдържат съответните битове. Паметта е организирана в йерархия от гледна точка на нейния обем и скорост на достъп до информацията. Тя се разбива на два големи класа: основна памет (RAM = SRAM + DRAM) и външна памет (магнитни дискове, CD, DVD). SRAM (Static RAM) или още наречена кеш памет осъществява много бърз достъп до най-често използваните данни и команди

Сегментна и странична преадресация

Преадресацията (адресиране) в компютърните системи представлява процесът на преобразуване на логическите (виртуалните) адреси в реални физически адреси в паметта. Този процес се реализира чрез различни методи като сегментна преадресация и странична преадресация.

Сегментна преадресация:

- сегмент - логически блок от паметта, съдържащ данни или инструкции, на които се разчита при изпълнението на програмата
- селектор - част от логическия адрес, която указва кой сегментен регистър да се използва за адресиране на паметта. Селекторът съдържа индекс към таблица на сегментите, която съдържа информация за всеки сегмент
- дескриптор - описание на даден сегмент, съдържащ началния адрес на сегмента в паметта, дължината на сегмента, правата за достъп и други атрибути. Дескрипторите се съхраняват в таблици на сегментите
- таблици на сегментите - таблици, които съхраняват дескрипторите на сегментите. Има два основни вида таблици на сегментите:
 - глобална таблица на сегментите - съдържа дескриптори за всички сегменти, които могат да се използват в системата
 - локална таблица на сегментите - съдържа дескриптори, които са специфични за текущия процес

- регистри на сегментите - специални регистри, които съдържат селектори и се използват за адресиране на сегментите

Процесът на сегментно адресиране се състои в следното:

1. Логическият адрес се състои от селектор и офсет
2. Селекторът посочва конкретен дескриптор в таблицата на сегментите
3. Дескрипторът съдържа базовия (началния) адрес на сегмента и неговата дължина
4. Офсетът указва конкретното изместване в рамките на сегмента
5. Крайният физически адрес се изчислява като сбор от базовия адрес на сегмента и офсета

Странична преадресация:

- страница - малък блок от паметта с фиксиран размер (например 4 KB). Виртуалната памет се разделя на страници
- каталог на страниците - съдържа списък с указатели към таблиците на страниците. Всеки процес има свой каталог на страниците
- таблици на страниците - всяка таблица на страниците съдържа указатели към физическите страници в паметта
- описател на страниците - съдържа информация за състоянието на страницата, права за достъп и физическия адрес, където се намира страницата в паметта
- стратегии за подмяна на страниците - поради ограничението на физическата памет, съществуват някои основни стратегии за подмяна на страниците:
 - FIFO (First-In, First-Out) - първата влязла страница е първата, която се премахва
 - LRU (Least Recently Used) - страница, която не е била използвана най-дълго време, се премахва
 - NRU (Not Recently Used) - подобно на LRU, но използва по-проста логика за избор на страниците

Процесът на странично адресиране се състои в следното:

1. Виртуалният адрес се разделя на три части: индекс към каталога на страниците, индекс към таблицата на страниците и офсет
2. Индексът към каталога на страниците указва конкретен вход в каталога, който сочи към таблица на страниците
3. Индексът към таблицата на страниците указва конкретен вход в таблицата на страниците, който сочи към физическата страница

4. Офсетът посочва конкретното изместване в рамките на физическата страница
5. Крайният физически адрес се получава чрез комбиниране на базовия адрес на страницата и офсета

Система за прекъсване

Системата за прекъсвания е ключова част от управлението на съвременните компютърни системи. Тя позволява на процесора да бъде информиран за важни събития и да реагира на тях, като прекъсва текущото изпълнение на програма и изпълнява специален код за обработка на съответното събитие.

Видове прекъсвания:

- Хардуерни:
 - външни прекъсвания - генерирани от външни устройства като клавиатура, мишка, таймери, мрежови карти и др.
 - вътрешни прекъсвания (изключения) - възникват в резултат на грешка при изпълнението на програма, като деление на нула, защита на паметта (например достъп до невалиден адрес), привилегировани операции и др.
- Софтуерни:
 - системни извиквания (system calls) - специални инструкции, използвани от програми за искане на услуги от операционната система, например вход/изход операции
 - програмни прекъсвания - могат да бъдат създадени от програмата чрез изпълнение на определена инструкция, която предизвиква прекъсване (например, в x86 архитектурата това може да е инструкцията INT).

Структура и обработка на прекъсванията

1. Генериране на прекъсване - прекъсванията могат да бъдат генерирани от различни източници: хардуерни устройства, софтуерни изключения, таймери и други. Когато възникне прекъсване, контролерът на прекъсванията информира процесора
2. Запазване на състоянието - когато процесорът получи сигнал за прекъсване, той запазва текущото си състояние (например съдържанието на програмния брояч и състоянието на регистрите) в стека, за да може да продължи изпълнението след обработката на прекъсването
3. Обработка на прекъсването - процесорът преминава към изпълнение на код, наречен рутината за обработка на прекъсването (Interrupt Service Routine, ISR), която е предназначена да обработи конкретния тип прекъсване. В ISR се извършва необходимата работа, например

четене на данни от устройство, извършване на корекции в състоянието на системата и други

4. Възстановяване на състоянието и продължаване на изпълнението - след приключване на обработката на прекъсването, състоянието на процесора се възстановява от стека и изпълнението на предходната програма продължава от мястото, където е било прекъснато.

Конкурентност - в една система може да възникнат множество прекъсвания едновременно или почти едновременно. За да се избегне объркване и загуба на данни, е необходимо прекъсванията да се обработват по приоритет или последователно. Някои прекъсвания могат да бъдат забранени временно, докато друго прекъсване се обработва, за да се предотврати прекъсване на ISR (което може да доведе до загуба на данни или нежелано поведение).

Приоритети - прекъсванията имат приоритети, които определят реда, в който трябва да бъдат обработвани. Например, прекъсване от таймер, който управлява ключови времеви операции в системата, може да има по-висок приоритет от прекъсване за вход/изход. Ако две прекъсвания възникнат едновременно, прекъсването с по-висок приоритет ще бъде обработено първо.

Контролер на прекъсванията (PIC - Programmable Interrupt Controller) - специализирано устройство, което управлява и приоритизира прекъсванията преди те да бъдат обработени от процесора. PIC често е отговорен за приемане на прекъсванията от различни устройства, присвояване на приоритети на различни прекъсвания, информироване на процесора за наличието на прекъсване и подаване на прекъсванията в правилния ред, изпращане на сигнали за забрана (masking) на определени прекъсвания, ако те не трябва да бъдат обработени веднага.

Операционни системи

11. Файлова система. Логическа организация и физическо представяне.

Файловата система е тази компонента на операционната система, която е предназначена да управлява постоянните обекти данни, т.е. обектите, които съществуват по-дълго отколкото процесите, които ги създават и използват. Постоянните обекти данни се съхраняват на външна памет (диск или друг носител) в единици, наричани файлове. Файловата система трябва да:

- осигурява операции за манипулиране на отделни файлове - create, open, close, read, write, delete и др.
- изгражда пространството от имена на файлове и да предоставя операции за манипулиране на имената в това пространство.

Логическа организация на ФС

Най-важната характеристика на една абстракцията за потребителите са правилата за именуване на обектите. Името на файл е низ от символи с определена максимална дължина, като в някои системи са разрешени и други символи. Обикновено името се състои от две части, разделени със специален символ, например ".". Втората част се нарича разширение на името и носи информация за типа или формата на данните, съхранявани във файла. Например:

- file.c - програма на Си
- file.o - програма в обектен код
- file.exe - програма в изпълним код
- file.txt - текстов файл

Какво включва пространството от имена или какви са типовете файлове? Преди всичко имена на **обикновени файлове (regular files)**, съдържащи програми, данни, текстове или каквото друго потребителят пожелае. Но пространството от имена би могло да включва и имена на външни устройства, системни структури и услуги. Външните устройства са специален тип файлове, наречени **специални файлове (character special и block special device file)** в Unix, Linux и др. Системните примитиви на файловата система са приложими както към обикновените файлове така и към специалните файлове. Следователно, всяка операция за четене или писане, осъществявана от програмата, е четене или писане във файл. Най-същественото предимство на този подход е, че позволява да се пишат програми, които не зависят от устройствата, тъй като действията, изпълнявани от програмата зависят само от името на файла. Трябва да се съхранява информация за файловете във файловата система и за тази цел се използват специални системни структури, наречани **каталог, справочник, директория, directory, folder, VTOC**, които осигуряват връзката между името и данните на файла и реализират организацията на файловете. В много системи каталогът е тип файл с фиксирана структура на записите и съдържа по един запис за всеки файл. И така, типовете файлове, поддържани от файловите системи на Unix, Linux и др. са:

- обикновен файл (regular file)
- каталог (directory)
- специален файл (character special file / block special device file)
- програмен канал, FIFO файл (named pipe / FIFO file)
- символни връзки (symbolic links)

Типът **програмен канал (pipe)** и **FIFO файл** се използва като механизъм за комуникация между конкурентни процеси. Чрез **символните връзки (symbolic link, soft link)** един файл може да има няколко имена,

евентуално в различни каталози, или както се казва реализират се няколко връзки към файл. Този тип файл е един от механизмите за осигуряване на общи файлове за различните потребители. Друг начин за работа с общи файлове са **твърдите връзки (hard links)**, но те не са тип файл, а са няколко имена на обикновен файл. Тясно свързан с въпроса за типовете файлове е вътрешната структура на файл. Файлът най-често представлява последователност от обекти данни. Какви са тези обекти данни? Възможни са два отговора - запис или байт: Файлът е последователност от записи с определена структура и/или дължина. Основното в този подход е, че всеки системен примитив read или write чете или пише един запис. Другата възможност, реализирана в много от съвременните операционни системи, Unix, Linux, MSDOS, Windows и др., е последователност от байтове. Това е структурата на файл, реализирана от файловата система. Всяка по-нататъшна структура на файла може да се реализира от програмите на потребителско ниво, като операционната система не помага за това, но и не пречи. Този подход позволява разработката на прости системни примитиви, които освен това да са приложими както за обикновени файлове, така и за останалите типове файлове. Всеки файл има име и данни. В допълнение операционната система може да съхранява и друга информация за файла, която ще наричаме атрибути на файла. Атрибутите, реализирани в различните системи се различават, но някои общи са: време на създаване, време на достъп, време на промяна, собственик, права на достъп, парола за достъп и флагове като read-only, system, archive и др.

Йерархична организация на файловата система

Вътрешните възли на дървото трябва да са каталози, а листата могат да бъдат каталози, обикновени файлове, специални файлове и други типове файлове, ако се поддържат такива. За потребителя каталогът представлява група от файлове и каталози (подкаталози). Този подход, използван при файловите системи на повечето съвременни операционни системи Unix, Linux, MSDOS, Windows и др., дава възможност за естествено и удобно групиране на файловете, отразяващо предназначението и формата на данните, съхранявани в тях. Всеки връх в дървото, каталог или друг тип файл, има име, което потребителят избира да е уникално в рамките на съдържащия го каталог и което ние ще наричаме собствено име. Тогава всеки файл ще има име, което уникално го идентифицира в рамките на цялата йерархична структура и ние ще го наричаме пълно име. Използват се два начина за формиране на пълно име:

- абсолютно пълно име (absolute path name) - всеки файл или каталог притежава едно абсолютно пълно име, което съответства на единствения път в дървото от корена до съответния файл или каталог
- относително пълно име (relative path name) - този начин за формиране на пълно име е свързан с понятието текущ или работен каталог (current/working directory). Във всеки един момент от работата на

потребителя със системата, той е позициониран в един от каталозите на дървото. Операционната система предоставя средства за избор на текущ каталог. Относителното пълно име на файл или каталог съответства на пътя в дървото от текущия в даден момент каталог до съответния файл или каталог. Като в този случай е разрешено и движение нагоре по дървото.

Пълното име на файл, абсолютно или относително, се състои от компоненти - собствените имена на каталозите в пътя и на самия файл, разделени със специален символ-разделител, като напр. >, \, /. Движението нагоре по дървото се записва чрез специално име, което обикновено е "..". Ако първият символ в пълното име е символа-разделител, това означава, че името е абсолютно пълно. Всяко име, което не започва със символа-разделител, се приема за относително пълно или собствено име. Абсолютно пълно име на файл е /home/ivan/letter. Ако текущ каталог е /home, то относителното пълно име на същия файл е ivan/letter. Ако текущ каталог е /home/emi, то относителното пълно име на същия файл е ../ivan/letter. Следователно, ако от текущ каталог /home/emi искаме да създадем копие на файла под име copyletter, имаме различни възможности за именуване на оригиналния файл и копието:

```
cp /home/ivan/letter /home/emi/copyletter
cp /home/ivan/letter copyletter
cp ../ivan/letter copyletter
```

Физическа организация на ФС

Стратегиите за управление на дисковото пространство са най-често две:

Статично и непрекъснато разпределение

За файл с размер n байта се отделя една непрекъсната област от диска, в която могат да се съхранят всичките n байта и това се прави при създаването на файла. За тази цел обаче, системата трябва да има предварителна информация за максималния размер на файла, който той може да достигне по време на съществуването си. Естествено тази информация може и трябва да се осигури от потребителя по време на създаване на файла и тогава системата ще разпредели дискова памет за файла. Преимуществото, което този метод дава, е високата производителност на системата при последователна обработка на файла, тъй като последователните байтове на файла са разположени в съседни сектори, писти и цилиндри на диска. Проблем може да възникне при нарастване на файла, което е нещо обичайно, ако размерът на файла надмине разпределената му предварително памет. Едно възможно решение на този проблем е разпределяне на нова непрекъсната област с по-голям размер и преместване на файла в новата област. Това обаче не е добро решение, тъй като операцията може да се окаже бавна и скъпа при големи файлове. Друго решение е да се направи компромис при искането за непрекъснатост на дисковата памет, т.е. един файл да може да заема не ед-

на, а няколко, но малко на брой непрекъснати области от диска с размери, заявени от потребителя. Такава стратегия е реализирана в операционната система OS/360 за машини от семейството на IBM/360. Например, ако при изпълнение на програма, създаваща файл, потребителят укаже в съответния JCL оператор:

```
DD . . . SPACE=(TRK, (20, 5)), . . .
```

то първоначално за файла се отделя една непрекъсната област от диска с размер 20 писти и това е така наречения първичен екстент. При запълване на тази област системата извършва вторично разпределение - разпределя втора непрекъсната област от диска с размер 5 писти. При продължаващо нарастване на файла вторичното разпределение се повтаря, но максимално 15 пъти. Следователно максималният размер на файл се ограничава от един първичен и 15 вторични екстента с размери избрани от потребителя. След това проблемът с нарастване на файла отново се появява. Друг недостатък на тази стратегия е известен като фрагментация на свободната дискова памет. Това означава съществуване на достатъчно свободни участъци дисковата памет, които обаче не са съседни и поради това не може да бъде удовлетворена заявка за първично или вторично разпределение на памет за файл. Причината за този проблем е изискването за непрекъснатост и нефиксирания размер на единиците за разпределение на дискова памет. Тази стратегия е използвана в по-старите операционни системи. Но в последните години с развитието на технологията и появата на CD-ROM и други еднократно записвани носители, статичното и непрекъснато разпределение отново се оказва приложимо. При създаване на CD-ROM диск размерите на файловете са предварително известни и не се изменят при последващо използване на файловата система.

Динамично и блоково разпределение

Файлът се дели на части с фиксирана дължина, които ще наричаме блокове. Последователните блокове на файла при записването им на диска не е задължително да са физически съседни. Това дава възможност дискова памет за файл да се разпределя по блокове тогава, когато е необходима, т.е. динамично при нарастване на файла. Така се решава и проблема с нарастване на файла и проблема с фрагментацията на свободната дискова памет, тъй като памет се разпределя динамично по блокове, които са с еднакъв фиксиран размер. При прилагане на тази стратегия трябва се избере размер на блока. Отчитайки организацията на диска, естествени кандидати са сектор, pista, цилиндър. Избор на голям блок, напр. цилиндър, би довел до неефективно използване на дисковата памет за сметка на частично запълнен последен блок на файл. Избор на малък блок пък означава, че големите файлове ще се състоят от много блокове, които е възможно да са несъседни и пръснати по дисковата повърхност. А това означава неефективност при последователна обработка на файла. Необходим е компромис, който най-често е 1К байта, 2К байта, 4К байта. Ако дисковият сектор е 512 байта, то всеки блок ще заема 2, 4 или 8 последователни сектора. При

по-нататъшното изложение ще предполагаме, че се използва динамично и поблоково разпределение на дискова памет, както е в повечето съвременни системи. Единицата за разпределение на дискова памет ще наричаме блок, въпреки че в някои операционни системи се използват и други термини, като зона, клъстер или тя е просто дисков сектор. Следователно, за файловата система дисковото пространство е последователност от блокове с фиксиран размер и адреси (номера) 0,1,2, ...N.

Системни структури

Файловата система трябва да съхранява информация за разпределението на дисковата памет, а именно за свободните блокове и за блоковете разпределени за всеки един файл. Под системни структури (наричат ги също метаданни) тук ще разбираме структури, съхраняващи такава информация постоянно на диска. Информация за всички свободни блокове трябва да бъде съхранявана, тъй като само по съдържанието на блока не може да се отличи свободния от заетия блок. Някои възможни структури данни, използвани за тази цел са следните.

Свързан списък на свободните блокове

Всички свободни блокове са организирани в едносвързан списък, т.е. първата дума от всеки свободен блок съдържа адрес на следващ свободен блок. Тази структура е реализирана във файловата система на UNIX. Недостатък на този метод е, че системната структура, т.е. свързаният списък, е пръсната по всички свободни блокове, което крие опасности за повредата ѝ при системни сривове.

Свързан списък на блокове с номера на свободни блокове

Свободните блокове се групират и номерата на блоковете от една група се записват в първия свободен блок. Следователно информацията се съхранява в едносвързан списък от дискови блокове, които съдържат номера на свободни блокове. Самите блокове от списъка вече не са свободни за разлика от предходната структура. Такава структура е по-компактна, големината ѝ е пропорционална на свободното дисково пространство и позволява разработката на ефективни алгоритми за разпределяне на блокове. Този подход е използван във файловата система на UNIX System V (s5fs).

Свързан списък на блоковете на файла

Всеки блок на файла ще съдържа в първата си дума номер на следващ блок на файла и данни в останалата част. Основният недостатък на тази структура е неефективната реализация на произволен достъп до файла. За да се прочете произволен байт от файла системата трябва да прочете всички предходни блокове в списъка докато стигне до данните, искани

от програмата. Друг недостатък е, че адресната информация е пръсната по всички блокове на файла, а това прави файловата система неустойчива при повреди. И още един недостатък, който в някои случаи е критичен, е че броят на байтовете за данни в блока вече не е степен на 2.

Общи параметри

На всеки диск или дял от диск (ако е разделен на дялове) се изгражда файлова система с определена физическа структура, обикновено наричана том (volume). Друг тип информация, която трябва се съхранява, са общи параметри на физическата структура на файловата система, т.е. на тома. Обикновено системната структура се нарича суперблок. В суперблока се съхраняват общи параметри, като:

- размер на файловата система (максимален номер на блок на тома)
- размер на блок
- размери и адреси на различни области от тома, съдържащи системни структури, като напр., на индексната област, на битовата карта, на FAT
- общ брой свободни блокове на тома и други ресурси, напр., индексни описатели.

12. Управление на процеси и междупроцесни комуникации.

Ще разгледаме системните примитиви по стандарта POSIX, които реализират основните операции за процеси, а именно създаване на процес, завършване на процес и други.

1.Създаване на процес

```
1 #include <sys/types.h>
2 pid_t fork(void);
```

Единственият начин да се създаде процес в Unix/Linux системите е чрез `fork`. Процесът, който изпълнява `fork` се нарича процес-баща, а новосъздаденият е процес-син. При връщане от `fork` двата процеса имат еднакви образи, с изключение на връщаната стойност: в процеса-баща `fork` връща `pid` на процеса-син при успех или `-1` при грешка, а в сина връща `0`. Сложността при този примитив се състои в това, че един процес "влиза" във функцията `fork`, а два процеса "излизат от" `fork` с различни връщани значения. Да разгледаме по-подробно това, което става когато процес-баща изпълнява `fork` и така ще разберем какво е общото между процесите баща и син.

Алгоритъм на `fork`

1. Определя уникален идентификатор за новия процес и създава запис в таблицата на процесите, като инициализира полетата в него (група и сесия от процеса-баща, състояние "новосъздаден" и т.н.).
2. Създава U агеа, където повечето от полетата се копират от процеса-баща (файловите дескриптори, текущ каталог, управляващ терминал, потребителските идентификатори - euid, ruid, egid и rgid).
3. Създава образ на новия процес - копие на образа на процеса-баща (регионът за код обикновено е общ за двата процеса).
4. Създава динамичната част от контекста на новия процес: Слой 1 е копие на слой 1 от контекста на бащата. Създава нов слой 2, в който записва съхранените регистри от работата в слой 1, като регистър РС е изменен така, че синът да започне изпълнението си в fork от стъпка 7.
5. Изменя състоянието на процеса-син в "готов в паметта".
6. В процеса-баща връща pid на новосъздадения процес-син.
7. В процеса-син, връща 0.

Следователно, когато по-късно планировчикът избере новосъздадения процес за текущ той ще заработи според най-горното ниво на динамичната част от контекста си (слой 2) - в системна фаза на fork и ще върне 0. И така процесите син и баща имат следното общо помежду си: - Двата процеса изпълняват една и съща програма. Дори процесът-син, който преди това не е работил, започва изпълнението на потребителската програма от оператора след fork. Но връщаните стойности в двата процеса са различни, което позволява да се определи кой процес е бащата и кой сина и да се раздели тяхната функционалност макар, че изпълняват една и съща програма. - Процесът-син наследява от бащата файловите дескриптори, т.е. двата процеса ще разделят достъпа до файловете, които бащата е отворил преди да изпълни fork. Това позволява пренасочване на входа и изхода и свързване на процеси с програмни канали. - Двата процеса имат един и същ текущ каталог, управляващ терминал, група процеси и сесия. - Двата процеса имат еднакви права - реален и ефективен потребителски идентификатори, което гарантира неизменност на привилегиите на потребител при работа в системата.

Има два начина за използване на fork. Процес иска да създаде свое копие, което да изпълнява една операция, докато другото копие изпълнява друга, например при процеси сървери. Процес иска да извика за изпълнение друга програма, тогава първо създава свое копие, след което в едното копие (обикновено в процеса-син) се извиква другата програма. Това се използва от командния интерпретатор.

2.Завършване на процес

```
1 void exit(int status);
```

Системата не поставя ограничение за времето на съществуване на процес. Има процеси, например процес init (с pid 1), които съществуват вечно в смисъл от boot до shutdown. Когато процес завършва той изпълнява exit.

Процесът може явно да го извика или неявно в края на програмата, но може и ядрото вътрешно да извика `exit` без знанието на процеса, например когато процеса получи сигнал, за който не е предвидил друга реакция. Значението на аргумента `status` е кода на завършване, който се предава на процеса-баща, когато той се интересува. Този системен примитив не връща нищо, защото няма връщане от него, винаги завършва успешно и след него процесът почти не съществува, т.е. той става зомби.

Алгоритъм на `exit`

1. Изпълнява `close` за всички отворени файлови дескриптори и освобождава текущия каталог.
2. Освобождава паметта, заемана от образа на процеса и потребителската област.
3. Сменя състоянието на процеса в зомби. Записва кода на завършване в таблицата на процесите. Ако процесът завършва по сигнал, код на завършване е номера на сигнала.
4. Урежда изключването на процеса от йерархията на процесите. Ако процесът има синове, то техен баща става процесът `init` и ако някой от тези синове е зомби изпраща сигнал "death of child" на `init`. Изпраща същия сигнал и на процеса-баща на завършващия процес.

3. Синхронизация със завършване на процеса-син

```
1 #include <sys/wait.h>
2 #include <sys/types.h>
3 pid_t wait(int *status);
```

Процес-баща може да синхронизира работата си със завършването на свой процес-син, т.е. да изчака неговото завършване ако още не завършил и да разбере как е завършил, чрез `wait`. Функцията на системния примитив връща `pid` на завършилия син, а чрез аргумента `status` кода му на завършване.

Алгоритъм на `wait`

1. Ако процесът няма синове, това е грешка и функцията връща -1.
2. Ако процесът има син в състояние зомби, т.е. синът вече е изпълнил `exit`, освобождава записа му от таблицата на процесите, като взема кода на завършване и неговия `pid` и ги връща.
3. Ако процесът има синове, но никой от тях не е зомби, той се блокира, като чака сигнал "death of child". Когато получи такъв сигнал, което означава, че някой негов син току що е станал зомби, продължава както в точка 2. Сега след като разгледахме системните примитиви `fork`, `exit` и `wait`, става по-ясно значението на състоянието зомби. След `fork` двата процеса - баща и син съществуват едновременно и се изпълняват асинхронно. Това означава, че бащата може да изпълни `wait` както преди така и след `exit` на сина.

Освен това не бива да се задължава процес-баща да изпълнява wait, той може да завърши веднага след като е създал син. Но тогава в системата ще останат вечни зомбита, които заемат записи в таблицата на процесите. Затова когато процес завършва неговите синове се осиновяват от процеса init, който се грижи за изчистване на системата от зомбита-сираци.

4.Изпълнение на програма

Когато с fork се създава нов процес, той наследява образа си от бащата, т.е. продължава да изпълнява същата програма. Но чрез exec всеки процес може да смени образа си с друга програма по всяко време от своя живот, както веднага след създаването си така и по-късно, дори няколко пъти в своя живот. За този системен примитив има няколко функции в стандартната библиотека, които се различават по начина на предаване на аргументите и използване на променливите от обкръжението на процеса.

```
1 int execl(const char *name, const char *arg0
2   [, const char *arg1]..., 0);
3 int execlp(const char *name, const char *arg0
4   [, const char *arg1]..., 0);
5 int execv(const char *name, const char *argv[]);
6 int execvp(const char *name, const char *argv[]);
```

Първият аргумент name указва към името на файл, съдържащ изпълним код, от който ще се създаде новия образ. При execvp и execlp name може да е собствено име на файл и тогава файлът се търси в каталозите от променливата PATH. В останалите случаи name трябва да е пълно име на файл. Останалите аргументи в execl и execlp са указатели към аргументите, които ще се предадат на функцията main когато новият образ започне изпълнението си. Във функциите execv и execvp има един аргумент argv, който е масив от указатели на аргументите за функцията main, който също трябва да завършва с елемент NULL.

Алгоритъм на exec

1. Намира файла, чието име е в аргумента name и проверява дали процесът има право за изпълнение на този файл.
2. Проверява дали файлът съдържа изпълним код.
3. Освобождава паметта, заемана от стария образ на процеса.
4. Създава нов образ на процеса, използвайки изпълнимия код във файла и копира аргументите на exec в новия потребителски стек.
5. Изменя значения на някои регистри в областта за съхранени регистри в слой 1 от динамичната част на контекста, напр. на PC, указател на стек. Така, когато процесът се върне в потребителска фаза ще заработи от началото на функцията main на новия образ.
6. Ако програмата е set-UID прави съответните промени на потребителски-

те идентификатори на процеса.

При успех, когато процесът се върне от `exec` в потребителска фаза, той изпълнява кода на новата програма, започвайки от началото, но това си остава същия процес. Не е променен идентификатора му, позицията му в йерархията на процесите дори голяма част от потребителската област, като файловете дескриптори, текущия каталог, управляващия терминал, групата на процесите, сесията. При грешка по време на `exec` става връщане в стария образ, така че функцията връща -1 при грешка, а при успех не връща нищо защото няма връщане в стария образ.

Разделянето на създаването на процес и извикването на програма за изпълнение в два системни примитива играе важна роля. Това дава възможност програмата на процес-баща да определя поне в началото функционалността на свой процес-син и например, да се реализира пренасочване на входа и изхода, и свързването на роднински процеси чрез програмни канали (заслуга за това има и наследяването на файловете дескриптори).

5. Информация за процес

```
1 pid_t getpid(void);
2 pid_t getppid(void);
```

Системният примитив `getpid` връща идентификатора на процеса, който го изпълнява, а `getppid` връща идентификатора на неговия процес-баща. И двата примитива винаги завършват успешно.

Примери:

```
1 #include <sys/types.h>
2 main(void) {
3     pid_t pid;
4     pid = fork();
5
6     if (pid < 0) { /* in parent process */
7         perror("Cannot fork");
8         exit(1);
9     }
10    else if (pid == 0) /* in child process */
11        printf("PID %d: Child started, parent is %d\n",
12              getpid(), /* Current (child) PID */
13              getppid()); /* Parent PID */
14    else { /* in parent process */
15        printf("PID %d: Started child PID %d\n",
16              getpid(), /* Current (parent) PID */
17              pid); /* Child PID */
18        sleep(3); /* Wait 3 seconds */
19    }
20    exit(0);
```

```

21 }

1 #include <sys/types.h>
2 main(void) {
3     pid_t pid;
4     int status;
5     pid = fork();
6
7     if (pid == 0) { /* in child process */
8         execl("/bin/ls", "ls", "-l", 0);
9         perror("Cannot exec ls");
10        exit(1);
11    }
12    else /* in parent process */
13        if (pid < 0) {
14            perror("Cannot fork");
15            exit(1);
16        }
17        else {
18            wait(&status);
19            printf("Parent after death of child:
status=%d.\n", status);
20            exit(0);
21        }
22 }

```

Междупроцесни комуникации

Семафори

През 1965г. Дейкстра предлага нов механизъм за междупроцесни комуникации и го нарича семафори (semaphores). С всеки семафор се свързва цяла променлива - брояч и списък на чакащи в състояние блокиран процеси. Значението на брояча е неотрицателно число. За семафорите Дейкстра определя три операции - инициализация, операция *P* и операция *V*. При инициализацията се създава нов обект семафор и се зарежда начално значение в брояча му, което е неотрицателно цяло число. Тази операция ще записваме като деклариране и инициализиране на променлива, например:

```

1 semaphore s = 1;

```

Операцията *P* проверява и намалява значението на брояча на семафора, ако това е възможно, в противен случай блокира процеса и го добавя в списъка на чакащи процеси, свързан със съответния семафор. Събитието, което блокирания процес чака, е увеличение на брояча на семафора. Това събитие ще настъпи когато друг процес изпълни операцията *V* над същия семафор. Операцията *V* събужда един блокиран по семафора процес, ако

има такива, а в противен случай увеличава брояча на семафора, т.е. запомня едно събуждане. За операциите P и V се използва и обозначението down и up. Алгоритмите на двете операции са следните:

```
1 P(s)
2 {
3     if ( s > 0 ) s = s - 1;
4     // else блокира процеса, изпълняващ P по семафора s;
5 }
6 V(s)
7 {
8     if (//има блокирани по семафора s процеси)
9         // събужда един процес;
10    else s = s + 1;
11 }
```

Същественото за двете операции е, че са неделими (атомарни), т.е. никой процес, изпълняващ $P(s)$ или $V(s)$ не може да бъде преразпределен и докато един процес изпълнява операция над семафор s друг процес не може да започне операция над s докато първата не завърши. Тази неделимост на операциите може да бъде осигурена при реализацията им в ядрото като системни примитиви. Модулите, реализиращи операциите, са част от ядрото на операционната система и там за осигуряване на взаимно изключване може да се използва забрана на прекъсванията или други апаратни средства за взаимно изключване.

Съобщения

За да комуникират два процеса чрез механизъм на съобщения между тях трябва да се установи комуникационна връзка (communication link). След това процесите обменят съобщения чрез два примитива:

- send(destination, message)
- receive(source, message)

Чрез *send* процес изпраща съобщение *message* към процес *destination*. Процес, който иска да получи съобщение, трябва да изпълни *receive*, при което съобщението се записва в *message*. Следователно, за да се осъществи предаване на едно съобщение между два процеса, е необходимо единият процес да изпълни *send*, а другият *receive*. В операционните системи има различни реализации на механизъм на съобщения.

Адресиране на съобщенията

Директна комуникация

Всеки процес именува явно процеса-получател или процеса-изпращач, т.е. *destination* и *source* са идентификатори/имена на процеси. За да се предаде съобщение от процес P към процес Q , трябва да се изпълнят примитивите.

```

1 // Процес Р (изпращач)      Процес Q (получател)
2   send(Q, message);         receive(P, message);

```

При този начин на адресация:

1. Комуникационната връзка се установява автоматично между двойката процеси, но всеки от тях трябва да знае идентификатора на другия.
2. Комуникационната връзка свързва точно два процеса.
3. Между всяка двойка процеси може да има само една комуникационна връзка.
4. Комуникационната връзка е двупосочна.

При този начин съществува симетрия при адресацията, всеки от процесите трябва да укаже идентификатора на другия процес. Може да се реализира асиметричен вариант на директна комуникация. В примитива `receive` се указва идентификатор на процес, както по-горе или може да е от вида:

```

1 receive(ANY, message);

```

Това означава, че процесът иска да получи съобщение от всеки, който му изпрати такова. Функцията ще върне идентификатора на процеса, от който е полученото съобщение. Косвена комуникация

Въвежда се нов обект, наричан пощенска кутия (mailbox), опашка на съобщенията (message queue) или порт (port). Съобщенията се изпращат в или се получават от пощенска кутия. Следователно, *destination* и *source* са идентификатори на пощенска кутия. За да се предаде съобщение от процес *P* към процес *Q*, трябва да се използва обща пощенска кутия, например с име *A* и да се изпълнят примитивите.

```

1 // Процес Р (изпращач)      Процес Q (получател)
2   send(A, message);         receive(A, message);

```

При този метод на адресация:

1. Комуникационната връзка се установява между процесите когато те използват обща пощенска кутия.
2. Комуникационната връзка може да свързва и повече от два процеса.
3. Между два процеса може да съществува повече от една комуникационна връзка.
4. Комуникационната връзка може да е едноръчна или двупосочна.

В този случай освен примитивите *send* и *receive* трябва да има и примитив за създаване на пощенска кутия. Също така възниква и въпросът за собствеността на пощенската кутия и за правата на процесите да изпращат съобщения в или да получават съобщения от определена пощенска

кутия. Друг въпрос е как се унищожава пощенска кутия. Една възможна реализация е собственик на пощенската кутия да става процесът, който я създаде. Всеки друг процес, който знае името на пощенската кутия и на който собственикът е дал права, може да я използва. Унищожаването на пощенска кутия може да се реализира чрез системен примитив, извикван явно от собственика ѝ. Друга възможност е процесите да могат да ползват обща пощенска кутия чрез механизма за създаване на процеси и когато последният процес, ползващ определена пощенска кутия, завърши тя да се унищожава автоматично от системата.

Компютърни мрежи

Компютърни мрежи и протоколи – OSI модел. Маршрутизация. Протоколи IPv4, IPv6, TCP, DNS.

OSI модел

Съвременните мрежови архитектури следват принципите на модела OSI (Open Systems Interconnection), създаден от международната организация по стандартизация ISO за връзка между отворени системи - системи, чиито ресурси могат да се използват от другите системи в мрежата. OSI моделът е абстрактен модел на мрежова архитектура, който описва предназначението на слоевете, но не се обвързва с конкретен набор от протоколи. Поради това OSI моделът се нарича още еталонен (опорен) модел и всъщност дава препоръки (Reference Model). В еталонния модел има 7 слоя – физически, канален, мрежов, транспортен, сесия, представителен, приложен. За разлика от OSI модела, в TCP/IP модела има 4 слоя - мрежов достъп, интернет, транспортен, приложен. TCP/IP е по-практически ориентиран и реално използван в мрежите, докато OSI е концептуален.

Разпределена маршрутизация

Обединената мрежа (internet) – това е съвкупност от няколко мрежи, наричани също така подмрежи (subnet), които се свързват помежду си с маршрутизатори. Организацията на съвместно транспортно обслужване в съставната мрежа се нарича междумрежово взаимодействие (internetworking). Една от основните функции на мрежовия слой е предаването на пакети между крайните възли в обединените мрежи (отделните подмрежи). Този слой на практика осъществява междумрежово свързване. Маршрут се нарича последователността от маршрутизатори, през които трябва да премине пакета от източника (изпращача) до целта (получателя). Задачата за избор на маршрут от няколко възможни се решава от маршрутизаторите и крайните възли на основание на маршрутните таблици. Записите в таблиците могат да се попълват ръчно или от протоколите за маршрутизация. Протоколите за маршрутизация (например RIP или OSPF) следва да бъдат отличавани от мрежовите протоколи (например IP или IPX). Докато първите събират и предават по мрежата само служебна информация за

възможните маршрути, вторите са предназначени за предаване на потребителски данни.

Мрежовите протоколи и протоколите за маршрутизация се реализират във вид на програмни модули на крайните възли (хостовете) и на междинните – маршрутизаторите. Маршрутизаторът представлява сложно многофункционално устройство (специализиран компютър), в задачите на което влиза:

- построяване на маршрутни таблици
- определяне на тяхна основа на маршрута
- буферизация
- фрагментация и филтрация на постъпващите пакети
- поддръжка на мрежовите интерфейси

Главните функции на маршрутизаторите са:

- избор на най-добрия път за пакетите до адреса на получателя
- комутация на приетия пакет от входящия интерфейс към съответстващия изходящ интерфейс

Маршрутизацията е процес за определяне на най-добрият път (маршрут), по който пакетът може да бъде доставен до получателя. Възможните пътища за предаване на пакетите се наричат маршрути. Най-добрите маршрути до известните получатели се съхраняват в маршрутната таблица. В зависимост от начина на попълване на маршрутните таблици се различават два вида маршрутизация:

- Статична маршрутизация - данните се въвеждат от мрежовия администратор
- Динамична маршрутизация - информацията постъпва от съседните маршрутизатори като се използва протокол за динамична маршрутизация

Маршрутизаторът оценява достъпните пътища до адреса на получателя и избира най-рационалния маршрут на база на някакъв критерий - метрика. Най-малката метрика означава най-добър маршрут. Метриката на статичен маршрут винаги е равна на 0. Процесът на маршрутизация на дейтаграмите се състои в определянето на следващия възел (next hop) по пътя на предаването на дейтаграмата и прехвърлянето ѝ към този възел, който се определя като цел или междинен маршрутизатор. Нито възелът изпращач, нито някой междинен маршрутизатор разполагат с информация за цялата верига, по която се предава дейтаграмата. Всеки маршрутизатор, а също така и възелът изпращач, базирайки се на адреса на получателя на дейтаграмата, определят единствено следващия възел от нейния маршрут.

Протоколите за маршрутизация се делят на:

- Външни - пренасят маршрутна информация между автономните системи (EGP, BGP)
- Вътрешни - прилагат се единствено в границите на определена автономна система. (RIP, OSPF)

Различават се следните класове протоколи за динамична маршрутизация:

- Протоколи с векторно разстояние (**Distance vector**) — протоколи за маршрутизация на база векторно разстояние, които използват за търсене на най-добрия път разстоянието до отдалечената мрежа. Всяко пренасочване на пакета от маршрутизатора се нарича hop (хоп, скок). За най-добър се счита пътят до отдалечената мрежа с най-малък брой хопове. Векторът определя направлението към отдалечената мрежа. Примери за протоколи за маршрутизация с векторно разстояние са RIP и IGRP.
- Протоколи със състояние на връзките (**Link state**) — обикновено се наричат "първият – най-краткия път"(SPF). Всеки маршрутизатор създава три различни таблици. Първата от тях проследява непосредственото свързване на съседите, втората — определя топологията на цялата обединена мрежа, а третата е маршрутната таблица. Устройството, действащо според протокол от типа състояние на връзките, има повече сведения за обединената мрежа, от колкото всеки протокол с векторно разстояние. Примери за протоколи за IP маршрутизация за състоянието на връзките са протоколите OSPF и IS-IS.

IPv4

Класова адресация:

- Разделя IP адресите на 5 класа - A, B, C, D, E според стойността на първия октет
- Класовете се разпознават по първите битове на адреса
- Основните класове са:
 - Клас A - за много големи мрежи (1.0.0.0 – 126.0.0.0)
 - Клас B - Клас B – за средни мрежи (128.0.0.0 – 191.255.0.0)
 - Клас C – за малки мрежи (192.0.0.0 – 223.255.255.0)
- Всеки клас има фиксирана мрежова и хостова част
- Недостатък е неефективно използване на адресното пространство (много адреси остават неизползвани, особено при клас A и B)

Безкласова адресация (CIDR):

- Въведена като подобрение с цел по-ефективно използване на IP адреси

- Няма фиксирани класове – мрежата се определя чрез маска на под-мрежата (subnet mask) или префикс (напр. /24)
- Позволява създаване на мрежи с произволен брой хостове (Пример: 192.168.1.0/24 – мрежа с 256 адреса)
- Използва се в съвременната IP маршрутизация
- Предимство е по-добрата скалируемост, ефективност и гъвкаво разпределение на адреси

IPv6

Основни характеристики на IPv6 протокол:

1. Дължината на IP адреса е увеличена до 16 байта, като така се предоставя на потребителите практически неограничено адресно пространство;
2. Опростена структура на заглавието, съдържащо само 8 полета (вместо 13 в IPv4 протокола). Последното позволява на маршрутизаторите по-бърза обработка на пакетите, т. е. повишава се тяхната производителност;
3. Подобрена поддръжка на незадължителни параметри, понеже в новото заглавие изискваните по-рано полета стават незадължителни, а измененият начин за представяне на незадължителните параметри ускорява обработването на пакетите от маршрутизаторите за сметка на отсъствието на обработване на тези параметри;
4. Подобрена система за безопасност. Автентификацията и конфиденциалността са основни характеристики на новия IP протокол;
5. Предвидена е възможност за разширяване на типовете (класовете) на предоставяните услуги, които могат да се появят в резултат на очаквания растеж на мултимедийния трафик. Отделено е по-голямо внимание на типа на предоставяните услуги. За тази цел в заглавието на пакета на IPv4 е било предвидено 8 разрядно поле.

TCP

TCP (протоколът за управление на предаването) е протокол от транспортния и сесийния слой на OSI/RM модела. TCP протоколът осигурява надеждно предаване на данните между приложните процеси, понеже използва установяване на логическо съединение между взаимодействащите процеси. Логическото съединение между два приложни процеса се идентифицира от двойка сокети (IP адрес, номер на порт), всеки от които описва един от взаимодействащите процеси. Информацията, постъпваща към TCP протокол в рамките на логическото съединение от протоколите от по-горния слой, се разглежда от TCP протокола като неструктуриран поток от байтове и

се записва в буфер. За предаването към мрежовия слой от буфера се изрязва сегмент, непревишаващ 64 KB (максималния размер на IP пакет). На практика дължината на сегмента се ограничаваше от стойността 1460 B, като така се осигурява той да се разположи в Ethernet кадър с TCP и IP заглавия. TCP съединението е ориентирано към пълно дуплексно предаване. Управлението на потока от данни в TCP протокола се осъществява с използването на механизма на плаващия прозорец с променлив размер. При предаването на сегмента възелът изпращач включва таймер и очаква потвърждение. Отрицателни квитанции не се изпращат, а се използва механизмът за таймаут. Възелът цел като получи сегмента формира и изпраща обратно сегмент с номер за потвърждение, равен на следващия пореден номер на очаквания байт. За разлика от много други протоколи, TCP протоколът потвърждава получаването не на пакети, а на байтове от потока. Ако времето за очакване на потвърждението изтече, изпращачът изпраща сегмента отново. Въпреки че изглежда прост в работата си протокол, в него има редица нюанси, които могат да доведат до някои проблеми. Първо, понеже сегментите при предаването по мрежата могат да се фрагментират, е възможна ситуация, при която част от предадения сегмент ще бъде приета, а останалата част ще се окаже изгубена. Второ, сегментите могат да пристигат във възела на получателя в произволен ред, което може да доведе до ситуация, при която байтовете от 2345 до 3456 вече са получени, но потвърждението за тях не може да бъде изпратено, понеже байтовете от 1234 до 2344 все още не са получени. Трето, сегментите могат да се задържат в мрежата толкова дълго, че при изпращача да изтече интервала за очакване на потвърждението и той да ги изпрати отново. Предаденият повторно сегмент може да премине по друг маршрут и може да бъде фрагментиран по друг начин, или сегментът може по маршрута да попадне случайно в претоварена мрежа. Като резултат за възстановяването на изходния сегмент ще се изисква достатъчно сложна обработка.

TCP осигурява своята надеждност чрез използването на следните механизми:

- Данните от приложението се разбиват на блокове (сегменти) с определен размер, които ще се изпращат.
- Когато TCP изпраща сегмент, той установява таймер, очаквайки, че от отдалечената страна ще пристигне потвърждение за този сегмент, че е получен. Ако потвърждението не е получено до изтичането на времето, сегментът се изпраща повторно.
- Когато TCP приема данните от отдалечената страна на съединението, то изпраща потвърждение. Това потвърждение не се изпраща незабавно, а обикновено след части от секундата.
- TCP осъществява изчисляване на контролна сума за своето заглавие и данните. Тази контролна сума се изчислява и в двата края на съединението, като целта ѝ е да се открият всякакви изменения в данните

по време на предаването им. Ако сегментът е с невярна контролна сума, TCP го отхвърля и потвърждение не се генерира. (Очаква се, че изпращачът ще отработи таймаута и ще направи повторно предаване.) Понеже TCP сегментите се инкапсулират във вид на IP дейтаграми, а IP дейтаграмите могат да са с нарушена последователност при предаването, също така и TCP сегментите могат да са с нарушена последователност при предаването. След получаването на данните TCP може при необходимост да измени тяхната последователност, като резултат приложението получава данните в правилен ред.

- Понеже IP дейтаграмата може да се дублира, приемащата страна на TCP трябва да отхвърли дублираните сегменти.
- TCP осъществява контрол на потока от данни. Всяка страна на TCP съединението има определено буферно пространство. TCP на приемащата страна позволява на отдалечената страна да изпраща данни само в този случай, ако получателят може да ги разположи в буфера. Последното предотвратява препълването на буферите на бавни хостове от бързи хостове.

TCP съединение

Съединението е съвкупност от информация за състоянието на потока от данни, включваща сокетите, номерата на изпратените, приетите и потвърдените октети, размерите на прозорците. Всяко съединение уникално се идентифицира в Интернет от двойката сокети. Съединението се характеризира за процеса с име, което представлява указател към TCB (Transmission Control Block) структура, съдържаща информация за съединението. Отварянето на съединението от процеса се реализира с извикването на функцията OPEN, на която се предава сокетът, с който трябва да се установи съединението. Функцията връща името на съединението. Различават се два типа отваряне на съединението: активно и пасивно. Работата с TCP протокол между двама абонати винаги преминава през следните три етапа: установяване (отваряне) на съединение, обмен на данни и затваряне на съединението.

Отваряне и затваряне на TCP съединение

За да се отвори (установи) TCP съединение, е необходимо да се извършат следните действия: 1. Страната - инициатор на съединението (клиент) изпраща SYN сегмент (вдигнат бит SYN в полето за флагове на заглавието на TCP), посочвайки името на домейна (или IPdst) и номера на порта на сървъра, към който клиентът иска да се присъедини, и началния номер на последователността на клиента (поле Sequence Number в заглавието на TCP). 2. Сървърът отговаря със сегмент SYN, съдържащ своя начален номер на последователността на сървъра заедно с вдигнат флаг SYN. Също така е вдигнат флаг ACK и е попълнено полето "номер на потвърждението" (Acknowledgement number), където се записва получението от клиента номер на последователността + 1. 3. Клиентът трябва да потвърди присти-

гането на SYN сегмента от сървъра с използването на ACK флага и с нова стойност в полето за потвърждение (полученият от сървъра номер на последователността + 1). Предаването на тези три сегмента е достатъчно за установяването на съединението (често този процес се нарича три стъпково ръкостискане, three - way handshaking). След това между страните (клиента и сървъра) е възможен двустранен обмен на данни по установеното съединение. При едностранно затваряне на съединението страната - инициатор на затварянето трябва да изпрати по установеното съединение FIN сегмент (с вдигнат флаг FIN), а също така и да получи ACK отговор от отдалечената страна с уведомление за получаването на FIN сегмента. След това съединението с възможност за двустранен обмен на данни преминава в еднопосочно състояние (едната страна е затворила съединението, а втората е активна и поддържа отворено съединението). За да се затвори напълно съединението, активната страна трябва да формира FIN сегмент и да получи за него потвърждение. Така при установяването на съединението на двете страни е необходимо да изпратят и да приемат 3 сегмента, а при затварянето му – 4.

DNS

DNS (Domain Name System, система за символни имена на домейните) е важна за работата на Интернет, понеже за свързването с отдалечената система е необходима информация за неговия IP адрес, а за хората е по-лесно да запомнят буквени (обикновено повече осмислени) адреси, отколкото последователността от цифри на IP адреса. Първоначално преобразуването между адреси на домейни и IP адреси се е реализирало с използването на специален текстов файл hosts, който се е създавал централизирано и автоматично се е разпращал на всяка от машините в своята локална мрежа. С растежа на Интернет е възникнала необходимост от ефективен, автоматизиран механизъм, какъвто е станала системата DNS. В TCP/IP стека се използва DNS, която има йерархична дървовидна структура, допускаща използване в името на домейна произволен брой съставни части. Съвкупността от имена, за които няколко от старшите съставни части съвпадат, образуват домейн (domain) от имена. Имената на домейните се назначават централизирано, ако мрежата е част от Интернет, иначе - локално. Съответствието между имената на домейните и IP адресите може да се установява както със средствата на локалния хост (с използването на файла hosts), така и с помощта на централизираната DNS система, основана на разпределена база за съответствие от вида <име на домейн – IP адрес>. Име на домейн е и символното име на компютъра.

DNS функционира по схемата "клиент-сървър". В качеството на клиентска част се използва процедурата за разрешаване на имената - resolver, а в качеството на DNS - сървър (BIND). DNS е разпределена система. Всеки DNS сървър съхранява имената на следващия слой от йерархията и освен таблицата на изобразяване на имената съдържа препратки към DNS сървърите на своите поддомейни, което опростява процедурата за търсене. За

ускоряване на търсенето на IP адреси в DNS сървърите се прилага процедура за кеширане на преминалите през тях отговори за определено време, обикновено от няколко часа до няколко дни.

Примери за имена на домейни за организации са:

- com - комерсиални организации
- edu - образователни организации
- gov - правителствени организации
- org - некомерсиални организации
- net - организации за поддръжка на компютърните мрежи

DNS притежава следните характеристики:

- Разпределеност на администрирането. Отговорността за отделните части от йерархическата структура се поема от различни хора или организации.
- Разпределено съхраняване на информацията. На всеки възел от мрежата задължително трябва да се пазят само тези данни, които влизат в неговата зона на отговорност и (възможно) адресите на кореновите DNS сървъри.
- Кеширане на информацията. Възелът може да съхранява някакво количество данни, които не са за собствената зона на отговорност. Целта е намаляване на натоварването на мрежата.
- Йерархична структура, в която всички възли са обединени в дърво, и всеки възел може или самостоятелно да определя работата на по-долу стоящите възли, или да ги делегира (предава) на други възли.
- Резервиране. За съхраняването и обслужването на своите възли (зони) отговарят (обикновено) няколко сървъра, разделени както физически, така и логически, като по този начин те осигуряват съхраняването на данните и отказоустойчивост (продължаване) на работата, даже в случай на отказ на един от възлите.

DNS протоколът използва в работата си TCP или UDP, с номер на порт 53 за отговори на заявките. Традиционно заявките и отговорите се изпращат във вид на една UDP дейтаграма. TCP се прилага за AXFR (full zone transfer) заявки. Обичайно този механизъм изпълнява репликация на информация на зоната между сървърите, но той също така може да се използва за злонамерени цели (като получаване на различна информация за осъществяване на спам, разпределени DoS атаки и т.н.). Когато се обръщаме към сървър с конкретно запитване (например с използване на домейни само от първо и второ ниво), тогава браузърът, използвайки resolver, изпълнява следния алгоритъм:

1. Търси запис с конкретното запитване във файла hosts, ако не открие, тогава
2. Изпраща запитване към известен DNS кеширащ сървър (като правило, локален), ако на този сървър такъв запис не е открит, тогава
3. DNS кеширащият сървър се обръща към DNS-ROOT сървъра със запитване за адреса на DNS сървъра, отговарящ за домейна от първо ниво. Ако получи адреса, тогава
4. DNS кеширащият сървър се обръща към DNS сървъра, отговарящ за домейна от първо ниво, със запитване за адреса на DNS сървъра, отговарящ за домейна от второ ниво. Ако получи адреса, тогава
5. DNS кеширащият сървър изпраща запитване към DNS сървъра, отговарящ за домейна от второ ниво. Ако получи адреса, тогава
6. DNS кеширащият сървър кешира адреса и го предава на клиента
7. Клиентът се обръща към открития IP адрес

Софтуерни технологии

26. Съвременни софтуерни технологии.

Дефиниция Софтуерен продукт наричаме всеки софтуер, разработен с цел да отговори на специфични потребности на даден потребител или да автоматизира определени задачи.

Дефиниция Софтуерен процес наричаме последователност от стъпки включващи дейности, ограничения и ресурси, които осигуряват постигането на някакъв вид резултат. По друг начин казано, процесът е съвкупност от процедури, организирани така че да се изграждат продукти, задоволяващи определени цели и стандарти.

Основни фази на софтуерен процес

- Комуникация
- Планиране
- Моделиране
- Конструирание
- Внедряване

Дефиниция Модел на софтуерен процес наричаме опростено описание на начина на разработване на софтуера, представено от определена гледна точка.

Софтуерни модели

1. **Модел на водопада (Waterfall)** - най-старият метод на структурирано разработване на софтуер, който предлага систематизиран, последователен подход към разработването на софтуер, който включва следните дейности: събиране на софтуерни изисквания, оценяване, изготвяне на график, проследяване, анализ и проектиране, генериране на код и тестване, доставяне и поддръжка.

Предимства:

- ясно разграничен процес, лесен за разбиране
- всяка стъпка завършва със създаване на множество от документи
- всяка дейност трябва да бъде напълно завършена, преди да се премине към следваща
- ясно са дефинирани входовете и изходите на дейностите, както и интерфейсите между отделните стъпки
- ясно са дефинирани ролите на разработчиците на софтуер

Недостатъци:

- моделът налага по-скоро структура на управлението на проект, отколкото насоки как се извършват отделните дейности
- произлязъл е от областта на хардуера и не отчита същността на софтуера като творчески процес на решаване на проблем
- потребителят трудно формулира всичките си изисквания в началото
- разделянето на проекта на отделни етапи не е гъвкаво (трудно е да се реагира на променящи се изисквания на клиента)

2. **Модел на бързата разработка (RAD)** - основата цел е кратък цикъл на разработка. Дейностите са комуникация, планиране (няколко софтуерни екипа работят паралелно), моделиране (на данните, на процес, бизнес моделиране), конструиране (използване на готови софтуерни компоненти и автоматизация на код) и внедряване.

Предимства:

- по-кратко време за разработка благодарение на паралелната работа на екипите
- по-добро взаимодействие с потребителите чрез бърза обратна връзка и итерации

Недостатъци:

- за големи приложения изисква много човешки ресурс

- когато функционалността на софтуерната система не може да бъде подходящо разделена в отделни модули
- когато е важна високата производителност на софтуерното приложение
- когато за разработката се разчита на нови и не достатъчно усвоени технологии

3. Еволюционни модели

- Постъпков (инкрементен) модел - системата не се доставя като едно цяло, а вместо това процесът на разработка и доставянето са разделени на стъпки, като всяка стъпка доставя само част от цялата функционалност. На идентифицираните потребителски изисквания се присвояват приоритети и тези с по-висок приоритет се реализират в първите стъпки. След като започне разработката на една стъпка, изискванията не се променят.
- Итеративен модел - в самото начало доставя цялостната софтуерна система, макар и част от функционалността да е в примитивна форма. При всяка следваща итерация не се добавя нова функционалност, а само се усъвършенства съществуващата.

Предимства:

- клиентът може да използва системата, преди да е готов целият продукт
- първите стъпки могат да служат като прототип, за да помогнат за извличане и изясняване на изискванията към следващите стъпки
- по-малък риск от неуспех на целия проект
- функционалностите от цялата система, които са с най-висок приоритет, са тествани най-много

Недостатъци:

- необходимостта от активно участие на клиентите по време на изпълнение на проекта може да доведе до закъснения
- уменията за комуникация и координация са от особено голямо значение при разработката и ако не са на достатъчно добро ниво, водят до проблеми
- неформалните заявки за подобрения след завършването на всяка стъпка могат да доведат до объркване
- тези модели могат да доведат до т.н. "scope creep"—бавно и постепенно разширяване на обхвата на приложението, без процесът да е сходящ.

4. Прототипен модел - създават се основни потребителски интерфейси, без да има някакво значително писане на код. Това означава, че се разработва

съкратена версия на системата, която изпълнява ограничено подмножество от функции. Използва се съществуваща система или компоненти от система, за да се демонстрират някои функции, които ще бъдат включени в разработваната система.

5. Спираловиден модел - еволюционен модел на софтуерен процес, който съчетава прототипния модел и модела на водопада. Движещият фактор е анализ на риска. Основните му характеристики са, че е итеративен/цикличесен подход и има множество от точки на прогреса.

Гъвкави (Agile) софтуерни технологии

Extreme Programming (XP) - най-известната и широко използвана гъвкава методология за разработване на софтуер, която поставя фокус върху гъвкавостта, бързата доставка на работещ софтуер и адаптирането към променящите се изисквания. Основните принципи и практики на Extreme Programming са:

- кратки цикли на разработка (short development cycles) - обикновено с продължителност 1-2 седмици
- непрекъснато тестване (continuous testing) - включва писането на автоматизирани тестове преди или по време на разработката на функционалностите (Test-Driven Development)
- рефакторинг на кода (code refactoring) - подобряване на структурата на кода без промяна на функционалността
- програмиране по двойки (pair programming) - практика, при която двама разработчици работят заедно на едно устройство като единият от тях пише кода, а другият го преглежда в реално време
- непрекъсната интеграция (continuous integration) - кодът се интегрира често в основната версия на проекта, като след всяка итерация се изпълняват всички тестове

SCRUM - популярен Agile метод за управление на софтуерни проекти, който се фокусира върху колаборативна работа в екип, итеративно развитие и адаптивност. Той е особено ефективен при сложни проекти, където изискванията могат да се променят по време на разработката. Основните концепции тук са:

- спринтове (итерации) - проектът се разделя на серии от кратки, фиксирани по време итерации, наречени спринтове (обикновено с продължителност от 1 до 4 седмици). Всяка итерация включва планиране, разработка, тестване и доставка на работещ част от софтуера
- продуктов беклог (product backlog) - списък с всички задачи, изисквания, функции, подобрения и корекции за даден продукт. Продукт-

товият собственик (product owner) управлява този списък, като го приоритизира според стойността за бизнеса и нуждите на клиента

- спринт беклог (sprint backlog) - по време на планирането на всеки спринт, екипът избира елементи от продуктивния беклог, които могат да бъдат завършени в рамките на спринта, т.е. включва задачите, които ще бъдат изпълнени, както и план за тяхното завършване
- ежедневни срещи (daily stand-ups) - кратка среща (обикновено 15 минути), провеждана всеки ден по време на спринта, на която екипът обсъжда напредъка, препятствията и планира работата за деня
- спринт ревю (sprint review) - в края на всеки спринт, екипът представя завършените задачи на заинтересованите страни. Това е възможност за получаване на обратна връзка и за адаптиране на продуктови изисквания, ако е необходимо
- ретроспектива (sprint retrospective) - след спринт ревюто, екипът провежда ретроспектива, в която обсъжда какво е работило добре, какво не е и как може да се подобри процесът в следващите спринтове

Дефиниция Функционални изисквания - описание на услугите, които системата трябва да предоставя, начинът по който системата трябва да реагира на конкретни входни данни и поведението и в конкретни ситуации.

Дефиниция Нефункционални изисквания - ограничения върху услугите или функционалността на системата като времеви ограничения, ограничения върху процеса на разработване, използваните стандарти и т.н.

Дефиниция Анализ на изискванията - проверка на извлечените изисквания дали правилно, точно, пълно, атомарно описват изискванията на клиента.

Софтуерни архитектури

27. Архитектури на софтуерни системи.

Дефиниция Архитектура на дадена софтуерна система е съвкупност от структури, показващи различните софтуерни елементи на системата, външно видимите им свойства и връзките между тях.

Софтуерните технологии, центрирани около софтуерната архитектура, се фокусират върху проектирането, разработката и управлението на цялостната структура на софтуерната система. Тези технологии са от решаващо значение за създаването на устойчив, модулен и лесен за поддръжка софтуер. Ето някои ключови софтуерни технологии и подходи, свързани със софтуерната архитектура:

- Архитектура на микроуслуги (Microservices Architecture) - Docker, Kubernetes
- Контейнеризация и оркестрация (Containerization and Orchestration) - Docker, Kubernetes
- Сервизно-ориентирана архитектура (SOA) - SOAP
- Архитектура, ориентирана към събития (Event-Driven Architecture) - Apache Kafka, RabbitMQ, AWS Lambda
- Слоеста архитектура (Layered Architecture) - MVC, MVP, MVVM
- Архитектура на базата на домейн (Domain-Driven Design) - Entity Framework, Hibernate
- Облачни архитектури - AWS, Azure, GCP

Качествени атрибути на софтуерната архитектура

- Ефективност – оптимално използване на ресурси (процесор, памет, мрежа); дизайнът трябва да позволява бързо изпълнение и ниска латентност
- Сложност – колко трудно се разбира, поддържа и разширява архитектурата. Добрата архитектура минимизира ненужната сложност
- Скалируемост – способност на системата да се справя с увеличаващо се натоварване (хоризонтално/вертикално мащабиране)
- Хетерогенност – възможност да се интегрират различни технологии, езици и платформи в една архитектура
- Адаптируемост – лесна промяна или добавяне на нова функционалност без сериозни преработки
- Надеждност – устойчивост на грешки и способност за възстановяване при отказ
- Сигурност – защита от неоторизиран достъп, атаки и изтичане на данни; включва криптиране, удостоверяване, контрол на достъпа

Дефиниция Компонент наричаме изчислителна (софтуерна) единица, която има определена функционалност и е достъпна чрез добре дефинирани интерфейси и има изрично специфицирани зависимости (входен и изходен интерфейс)

Дефиниция Конектор наричаме first class entity, представляващ механизъм за взаимодействие (протокол) между компонентите и правилата за комуникация (трансфер на данни)

Типове конектори и техните характеристики

Типове конектори:

- Пряко извикване (Direct invocation) – напр. API повиквания, методи, процедури
- Съобщения (Message passing) – синхронна или асинхронна комуникация (напр. с message queue)
- Потоци от данни (Data stream) – използват се при непрекъснато предаване на данни
- Общи данни (Shared data) – достъп до споделено хранилище или кеш
- Събитийни конектори (Event-based) – наблюдение и реакция на събития (event listeners)

Променливи характеристики на конекторите:

- начин на комуникация: синхронна vs асинхронна
- насоченост: еднопосочна или двупосочна
- тип пренос: текстов, двоичен, сериализиран обект
- прозрачност: дали скриват детайли за транспорта, формата и местоположението на компонентите
- устойчивост при грешки, сигурност, производителност

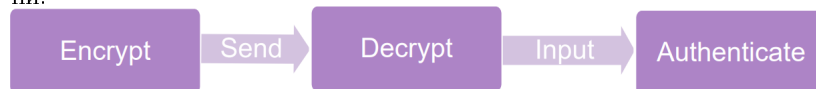
Критерии за избор на подходящи конектори

- видът на комуникацията между компонентите (нужна ли е синхронност?)
- честота и обем на обменяните данни
- изисквания за производителност и латентност
- сигурност и надеждност на връзката
- съвместимост с използваните технологии и среди

Дефиниция **Архитектурен стил** наричаме семейство от системи по отношение на модел (образец) на структурна организация.

Архитектурни стилове

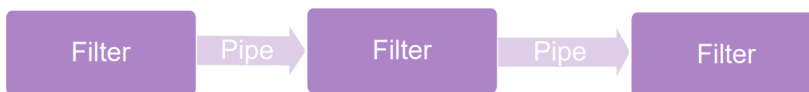
1. **Pipe-and-Filter** - всеки компонент в системата прехвърля данните в последователен ред към следващия компонент. Конекторите (pipes) между компонентите представляват действителните механизми за пренос на данни.



Името pipe-and-filter идва от системата Unix, където е възможно да се свържат процеси, използвайки тръби (pipes). Тръбите предават текстов поток от

един процес в друг и имат задължението да прехвърлят данните от изхода на филтър до входа на следващия филтър. Филтрите представляват изчислителни единици в системата - те съдържат функционалността и четат данни чрез входни интерфейси, обработват ги и ги изпращат до изходните интерфейси. Вариации на pipe-and-filter:

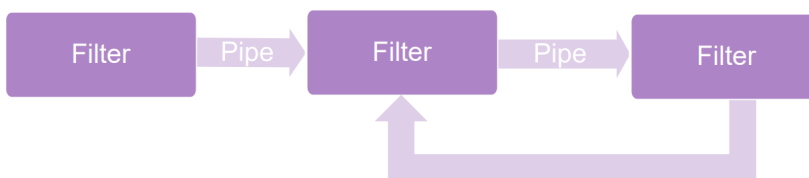
- Последователна обработка



- Паралелна обработка



- Цикъл (Loopback)



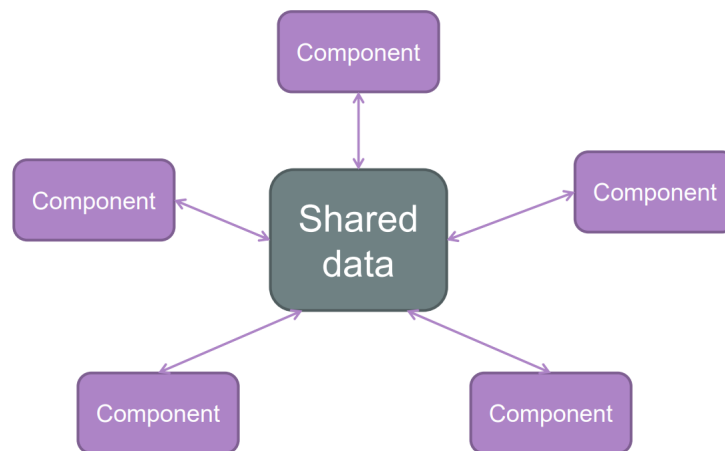
Предимства:

- интуитивен и лесен за внедряване
- филтрите са самостоятелни и могат да бъдат третираны като черни кутии, което води до гъвкавост по отношение на поддръжката и повторната употреба

Недостатъци:

- поради последователните стъпки на изпълнение е трудно да се реализират интерактивни приложения
- ниска производителност - всеки филтър трябва да анализира данните, трудно се споделят глобални данни и филтрите трябва да се съгласуват относно формата на данните

2. Shared data - активно се използва в системи, където компонентите трябва да прехвърлят големи количества данни. Споделените данни могат да се разглеждат като конектор между компонентите.



Вариации на shared data стила са:

- Blackboard (черна дъска) - когато се изпращат данни към конектора за споделени данни, всички компоненти трябва да бъдат информирани за това (с други думи споделените данни са активен агент)
- Repository (хранилище) - споделените данни са пасивни, до компонентите не се изпращат известия

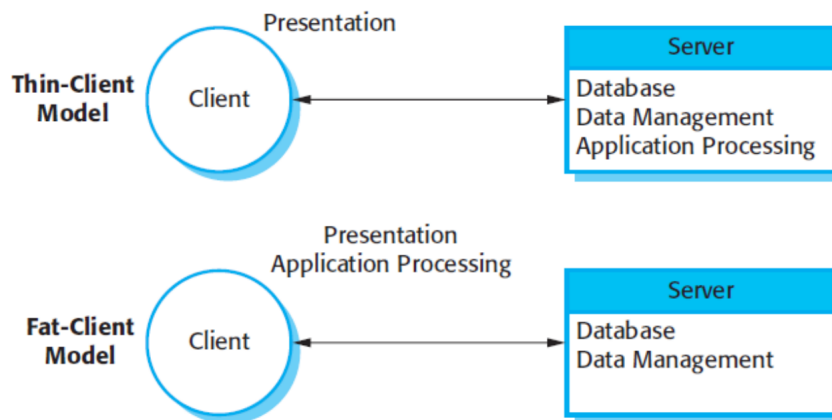
Предимства:

- скалируемост (могат да се добавят лесно нови компоненти)
- конкурентност (всички компоненти могат да работят паралелно)
- високо ефективен при обмен на големи количества данни
- централизирано управление на данните (по-добра сигурност, архивиране, т.н.)

Недостатъци:

- трудно се прилага в разпределена среда
- споделените данни трябва да поддържат единен модел на данни
- промените в модела могат да доведат до ненужни разходи
- може да се превърне в тясно място (bottleneck) в случай на твърде много клиенти

3. Client-server - системата е проектирана като набор от сървъри, които предлагат услуги и редица клиенти, които използват тези услуги. Тънък клиент (thin client) - клиентът реализира функционалността на потребителския интерфейс, а сървърът реализира функцията за управлението на данни и приложната обработка. Тежък клиент (fat client) - клиентът може да внедри част от функционалността за обработка на приложения.



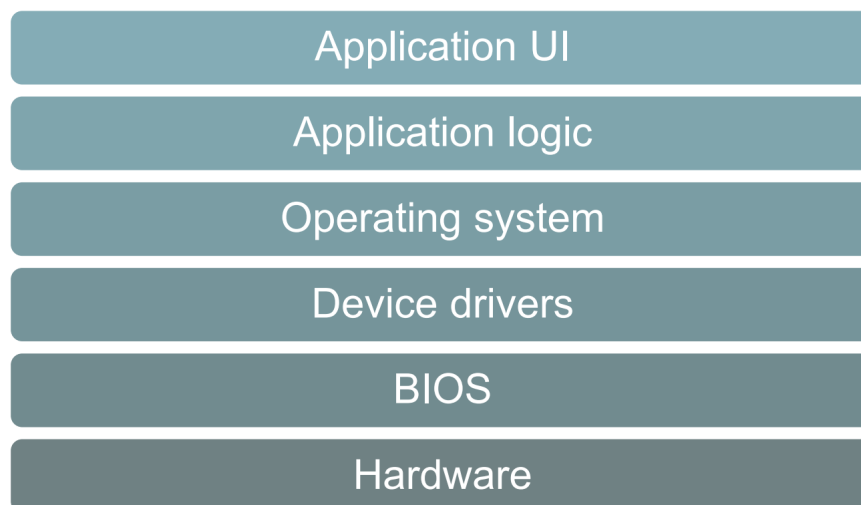
Предимства:

- централизация на данните
- сигурност - при клиента и при сървърите
- лесна реализация на архивиране (back-up) и възстановяване (recovery)

Недостатъци:

- работният товар на сървъра може да нарасне прекалено при голям брой клиенти
- необходимост от излишък/отказоустойчивост (redundancy, fault-tolerance)
- лесна реализация на архивиране (back-up) и възстановяване (recovery)

4. **Layered** - представя системата като организирана от йерархично-подредени слоеве



Предимства:

- вътрешната структура на слоевете е скрита, ако се поддържат интерфейси
- абстракция - минимизиране на сложността
- по-добра кохезия (всеки слой съдържа подобни задачи)

Недостатъци:

- за много системи е трудно да се разграничават отделните слоеве, което води до значителни усилия за проектиране
- строгите ограничения за комуникация на слоя компрометират производителността

5. Object-oriented